

第4章：Nginx 反向代理与负载均衡

一、学习目标

- 理解反向代理的概念和工作原理
 - 掌握 Nginx 反向代理的基本配置
 - 理解负载均衡的作用和常用算法
 - 掌握 upstream 配置方法
 - 学会配置健康检查和故障转移
-

二、理论知识

2.1 反向代理概念

什么是反向代理？

反向代理是服务器端的代理。客户端访问反向代理服务器，由代理服务器将请求转发给后端真实服务器，再将响应返回给客户端。客户端并不知道真实服务器的存在。

生活中的例子：你拨打某公司客服电话，接听的是总机，总机根据你的需求转接到不同的客服人员。你不知道具体是谁在服务你，总机就是反向代理。

反向代理的作用：

第一，负载均衡。将请求分发到多台后端服务器，提高系统处理能力。

第二，安全隔离。后端服务器隐藏在内网，不直接暴露给公网。

第三，SSL 终止。在代理层处理 HTTPS 加解密，减轻后端负担。

第四，缓存加速。缓存后端响应，减少重复请求。

2.2 负载均衡概念

什么是负载均衡？

当单台服务器无法处理所有请求时，需要多台服务器共同分担。负载均衡就是将请求合理地分配到多台服务器上，使每台服务器的负载相对均衡。

Nginx 支持的负载均衡算法：

算法	说明	适用场景
轮询（默认）	按顺序依次分配	后端服务器性能相近
加权轮询	按权重比例分配	服务器性能不同
IP Hash	相同 IP 访问相同服务器	需要会话保持
最少连接	分配给连接数最少的服务器	请求处理时间差异大

2.3 代理头信息

请求经过反向代理后，后端服务器收到的客户端 IP 会变成 Nginx 的 IP。需要通过 HTTP 头传递真实信息：

X-Real-IP	客户端真实 IP
X-Forwarded-For	代理链 IP 列表
X-Forwarded-Proto	原始请求协议 (http/https)
Host	原始请求的主机名

三、实践内容

3.1 实验环境准备

```
# 创建后端服务目录和页面
for i in 1 2 3; do
    sudo mkdir -p /var/www/backend$i
    sudo tee /var/www/backend$i/index.html << EOF
```

```

<!DOCTYPE html>
<html>
<head><meta charset="utf-8"><title>后端 $i</title></head>
<body style="font-family:Arial;text-align:center;padding:50px;background:#e3f2fd;">
<h1>后端服务器 $i</h1>
<p>端口: 900$i</p>
</body>
</html>
EOF
done

# 创建启动脚本
sudo tee /usr/local/bin/start-backends.sh << 'EOF'
#!/bin/bash
pkill -f "python3 -m http.server 900" 2>/dev/null
for i in 1 2 3; do
    cd /var/www/backend$i && nohup python3 -m http.server 900$i >
/dev/null 2>&1 &
done
echo "后端服务已启动: 9001, 9002, 9003"
EOF
sudo chmod +x /usr/local/bin/start-backends.sh

# 启动后端服务
sudo /usr/local/bin/start-backends.sh

# 验证后端
curl -s http://localhost:9001/ | grep "后端服务器"
curl -s http://localhost:9002/ | grep "后端服务器"
curl -s http://localhost:9003/ | grep "后端服务器"

```

3.2 实验一：基本反向代理

将所有请求代理到单个后端服务器。

```

# 创建配置
sudo tee /etc/nginx/conf.d/proxy-basic.conf << 'EOF'
server {
    listen 8080;

```

```

server_name localhost;

location / {
    # 代理到后端
    proxy_pass http://127.0.0.1:9001;

    # 传递客户端信息
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
}
EOF

# 测试并重载
sudo nginx -t && sudo systemctl reload nginx

# 开放端口
sudo firewall-cmd --permanent --add-port=8080/tcp
sudo firewall-cmd --reload

# 测试
curl http://localhost:8080/

```

核心指令说明：

proxy_pass	后端服务器地址
proxy_set_header	设置传递给后端的请求头

3.3 实验二：根据路径代理到不同后端

```

# 创建配置
sudo tee /etc/nginx/conf.d/proxy-path.conf << 'EOF'
server {
    listen 8081;
    server_name localhost;

```

```

# /app1/ 代理到后端1
location /app1/ {
    proxy_pass http://127.0.0.1:9001/;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
}

# /app2/ 代理到后端2
location /app2/ {
    proxy_pass http://127.0.0.1:9002/;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
}

# /app3/ 代理到后端3
location /app3/ {
    proxy_pass http://127.0.0.1:9003/;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
}

# 默认页面
location / {
    return 200 'Path Router: /app1/ /app2/ /app3/\n';
    add_header Content-Type text/plain;
}
}
EOF

# 测试并重载
sudo nginx -t && sudo systemctl reload nginx

# 测试不同路径
curl -s http://localhost:8081/app1/ | grep "后端服务器"
curl -s http://localhost:8081/app2/ | grep "后端服务器"
curl -s http://localhost:8081/app3/ | grep "后端服务器"

```

注意 proxy_pass 末尾斜杠的区别：

有斜杠: `location /app1/ + proxy_pass http://backend/
/app1/page.html → 后端收到 /page.html`

无斜杠: `location /app1/ + proxy_pass http://backend
/app1/page.html → 后端收到 /app1/page.html`

3.4 实验三：轮询负载均衡

使用 `upstream` 定义后端服务器组，实现负载均衡。

```
# 创建配置
sudo tee /etc/nginx/conf.d/lb-roundrobin.conf << 'EOF'
# 定义后端服务器组
upstream backend_servers {
    server 127.0.0.1:9001;
    server 127.0.0.1:9002;
    server 127.0.0.1:9003;
}

server {
    listen 8082;
    server_name localhost;

    location / {
        proxy_pass http://backend_servers;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
EOF

# 测试并重载
sudo nginx -t && sudo systemctl reload nginx

# 多次请求观察轮询效果
echo "=== 轮询测试 ==="
for i in {1..6}; do
    curl -s http://localhost:8082/ | grep -o "后端服务器 [0-9]"
```

done

预期输出：请求依次分配到后端1、2、3、1、2、3...

3.5 实验四：加权轮询

给不同服务器设置不同权重，性能好的服务器处理更多请求。

```
# 创建配置
sudo tee /etc/nginx/conf.d/lb-weight.conf << 'EOF'
upstream backend_weighted {
    server 127.0.0.1:9001 weight=5; # 50% 流量
    server 127.0.0.1:9002 weight=3; # 30% 流量
    server 127.0.0.1:9003 weight=2; # 20% 流量
}

server {
    listen 8083;
    server_name localhost;

    location / {
        proxy_pass http://backend_weighted;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
EOF

# 测试并重载
sudo nginx -t && sudo systemctl reload nginx

# 测试加权效果（10次请求）
echo "=== 加权轮询测试 ==="
for i in {1..10}; do
    curl -s http://localhost:8083/ | grep -o "后端服务器 [0-9]"
done | sort | uniq -c
```

预期输出：后端1出现约5次，后端2约3次，后端3约2次。

3.6 实验五：IP Hash（会话保持）

相同客户端 IP 始终访问同一后端，用于需要会话保持的场景。

```
# 创建配置
sudo tee /etc/nginx/conf.d/lb-iphash.conf << 'EOF'
upstream backend_iphash {
    ip_hash;
    server 127.0.0.1:9001;
    server 127.0.0.1:9002;
    server 127.0.0.1:9003;
}

server {
    listen 8084;
    server_name localhost;

    location / {
        proxy_pass http://backend_iphash;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
EOF

# 测试并重载
sudo nginx -t && sudo systemctl reload nginx

# 测试 IP Hash（同一客户端多次请求）
echo "=== IP Hash 测试 ==="
for i in {1..5}; do
    curl -s http://localhost:8084/ | grep -o "后端服务器 [0-9]"
done
```

预期输出：同一客户端的所有请求都到同一后端。

3.7 实验六：最少连接

将请求分配给当前连接数最少的服务器。

```
# 创建配置
sudo tee /etc/nginx/conf.d/lb-leastconn.conf << 'EOF'
upstream backend_leastconn {
    least_conn;
    server 127.0.0.1:9001;
    server 127.0.0.1:9002;
    server 127.0.0.1:9003;
}

server {
    listen 8085;
    server_name localhost;

    location / {
        proxy_pass http://backend_leastconn;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
EOF

# 测试并重载
sudo nginx -t && sudo systemctl reload nginx

# 测试
curl http://localhost:8085/
```

3.8 实验七：健康检查与故障转移

配置后端服务器的健康检查参数，当服务器故障时自动跳过。

```
# 创建配置
sudo tee /etc/nginx/conf.d/lb-health.conf << 'EOF'
upstream backend_health {
    server 127.0.0.1:9001 max_fails=3 fail_timeout=30s;
    server 127.0.0.1:9002 max_fails=3 fail_timeout=30s;
    server 127.0.0.1:9003 max_fails=3 fail_timeout=30s backup;
}
EOF
```

```
server {
    listen 8086;
    server_name localhost;

    location / {
        proxy_pass http://backend_health;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;

        # 超时设置
        proxy_connect_timeout 5s;
        proxy_read_timeout 10s;

        # 故障时尝试下一个服务器
        proxy_next_upstream error timeout http_500 http_502 http_503;
        proxy_next_upstream_tries 3;
    }
}
EOF
```

测试并重载

```
sudo nginx -t && sudo systemctl reload nginx
```

正常测试

```
echo "=== 正常状态 ==="
```

```
curl -s http://localhost:8086/ | grep "后端服务器"
```

模拟故障：停止后端1

```
echo ""
```

```
echo "=== 停止后端1，模拟故障 ==="
```

```
pkill -f "python3 -m http.server 9001"
```

```
sleep 2
```

再次测试（应该只返回后端2）

```
for i in {1..4}; do
```

```
    curl -s http://localhost:8086/ | grep -o "后端服务器 [0-9]"
```

```
done
```

恢复后端1

```
echo ""
```

```
echo "=== 恢复后端1 ==="
```

```
cd /var/www/backend1 && nohup python3 -m http.server 9001 > /dev/null  
2>&1 &
```

健康检查参数说明：

max_fails=3	失败3次后标记为不可用
fail_timeout=30s	30秒后重新尝试
backup	备份服务器，仅当主服务器全部故障时启用

故障转移参数说明：

proxy_next_upstream	触发转移的条件
proxy_next_upstream_tries	最多尝试几个服务器

3.9 实验八：综合配置示例

整合反向代理和负载均衡的最佳实践。

```
# 创建综合配置  
sudo tee /etc/nginx/conf.d/lb-complete.conf << 'EOF'  
# API 服务器组  
upstream api_servers {  
    least_conn;  
    server 127.0.0.1:9001 weight=3 max_fails=3 fail_timeout=30s;  
    server 127.0.0.1:9002 weight=2 max_fails=3 fail_timeout=30s;  
    server 127.0.0.1:9003 weight=1 max_fails=3 fail_timeout=30s  
    backup;  
  
    keepalive 32;  
}  
  
server {  
    listen 8087;  
    server_name localhost;  
  
    # 通用代理设置  
    proxy_set_header Host $host;
```

```
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
```

连接优化

```
proxy_http_version 1.1;
proxy_set_header Connection "";
```

超时设置

```
proxy_connect_timeout 5s;
proxy_read_timeout 60s;
proxy_send_timeout 10s;
```

故障转移

```
proxy_next_upstream error timeout http_500 http_502 http_503;
proxy_next_upstream_tries 2;
```

API 负载均衡

```
location /api/ {
    proxy_pass http://api_servers/;
}
```

健康检查

```
location /health {
    access_log off;
    return 200 "OK\n";
    add_header Content-Type text/plain;
}
```

默认页面

```
location / {
    proxy_pass http://api_servers;
}
```

```
}
```

```
EOF
```

测试并重载

```
sudo nginx -t && sudo systemctl reload nginx
```

测试

```
echo "=== 综合测试 ==="
```

```
for i in {1..6}; do
```

```
curl -s http://localhost:8087/ | grep -o "后端服务器 [0-9]"  
done
```

3.10 清理实验环境

```
# 停止后端服务  
pkill -f "python3 -m http.server 900"  
  
# 查看所有实验配置  
ls /etc/nginx/conf.d/proxy-*.conf /etc/nginx/conf.d/lb-*.conf  
2>/dev/null  
  
# 如需清理配置  
# sudo rm /etc/nginx/conf.d/proxy-*.conf /etc/nginx/conf.d/lb-*.conf  
# sudo systemctl reload nginx
```

四、本章小结

核心配置速查

基本反向代理：

```
location / {  
    proxy_pass http://backend;  
    proxy_set_header Host $host;  
    proxy_set_header X-Real-IP $remote_addr;  
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
}
```

负载均衡：

```
upstream backends {  
    # 算法: 默认轮询 / ip_hash / least_conn  
    server 127.0.0.1:9001 weight=3;  
    server 127.0.0.1:9002 weight=2;  
    server 127.0.0.1:9003 backup;  
}
```

健康检查:

```
server 127.0.0.1:9001 max_fails=3 fail_timeout=30s;  
proxy_next_upstream error timeout http_500;
```

知识要点总结

类别	要点
反向代理	隐藏后端、SSL终止、安全隔离
代理头	X-Real-IP、X-Forwarded-For、Host
负载均衡	轮询、加权、IP Hash、最少连接
健康检查	max_fails、fail_timeout、backup
故障转移	proxy_next_upstream

五、课后练习

1. 配置一个反向代理，将 `/api/` 路径代理到 API 服务器组，使用加权轮询
2. 配置 IP Hash 负载均衡，测试会话保持效果
3. 模拟后端故障，验证故障转移是否生效
4. 思考题：在什么场景下应该使用 IP Hash？什么场景下不适合？